Unidad I - LENGUAJES DE PROGRAMACIÓN ESTRUCTURADA, CARACTERISTICAS Y FUNDAMENTOS

Prologo

"La programación cambia diariamente". Una frase bastante cliché que representa una realidad que no muchos entienden, pero que esconde dentro de sí una oportunidad realmente provechosa para quien logre tomarla. Luego de quiza ver un lenguaje más o menos estructurado, más orientado a objetos o de paradigma más funcional, pues es momento de aprender sobre uno de los lenguajes de programación más importantes y más usados de hoy en día, como es Python, aprenderlo es una excelente forma de abrirse las puertas de muchas áreas de la programación.

Introducción

Python es uno de los lenguajes de programación más populares y versátiles del mundo, conocido por su simplicidad y legibilidad. Es una excelente opción tanto para programadores principiantes como para expertos que trabajan en proyectos complejos. Python fue creado por el programador neerlandés Guido van Rossum, quien comenzó a trabajar en él a finales de la década de 1980 como un proyecto personal. El nombre "Python" no proviene de una serpiente, sino de la afición de Van Rossum por el grupo de comedia británico Monty Python.

Python se destaca por una serie de características que lo hacen único y poderoso:

- **Lenguaje de Alto Nivel:** Esto significa que su sintaxis es más cercana al lenguaje humano y se aleja de los complejos detalles técnicos de la computadora.
- **Interpretado:** Python ejecuta el código línea por línea, lo que facilita la depuración de errores. A diferencia de los lenguajes compilados, no es necesario un paso de compilación antes de ejecutar el programa.
- **Sintaxis Simple y Legible:** A menudo se dice que el código Python es casi como leer inglés. Utiliza sangría en lugar de llaves para definir bloques de código, lo que obliga a mantener un código limpio y ordenado.
- **Multiparadigma:** Python soporta múltiples estilos de programación, incluyendo la programación orientada a objetos, la programación imperativa y, en menor medida, la funcional.
- **Tipado Dinámico:** No es necesario declarar el tipo de una variable (por ejemplo, si es un número o un texto) al escribir el código; Python lo determina automáticamente durante la ejecución.

- **Multiplataforma y de Código Abierto:** El código de Python puede ejecutarse sin modificaciones en diferentes sistemas operativos como Windows, macOS y Linux. Además, es de código abierto, lo que significa que es gratuito y cuenta con una gran comunidad global que contribuye a su desarrollo.

Usos Comunes de Python

La versatilidad de Python le ha permitido ser una herramienta fundamental en una gran variedad de campos. Algunos de sus usos más destacados incluyen:

Ciencia de Datos y Machine Learning: Es el lenguaje líder en este campo gracias a potentes bibliotecas como Pandas, NumPy, Scikit-learn y TensorFlow, que facilitan el análisis de datos, la visualización y la creación de modelos de inteligencia artificial.

Desarrollo Web: A través de frameworks como Django y Flask, Python permite construir el backend (la parte del servidor) de sitios y aplicaciones web robustas y escalables.

Automatización y Scripting: Es ideal para automatizar tareas repetitivas, como organizar archivos, extraer información de sitios web (web scraping) o realizar pruebas de software.

Desarrollo de Software: Se utiliza para crear aplicaciones de escritorio, juegos y otras herramientas de software.

Big Data y Blockchain: Su capacidad para manejar grandes volúmenes de datos y su seguridad lo hacen una opción viable para tecnologías emergentes.

1.1 Fundamentos

Indentación

La Indentación consiste en agregar espacios o tabulaciones al inicio de una línea de código para indicar que pertenece a un bloque específico. En Python, esta técnica define la estructura del programa, a diferencia de otros lenguajes que usan llaves {} para delimitar bloques.

En Python, cada bloque de código (como el contenido de una función, bucle o condicional) debe estar indentado. Si la indentación no es correcta, el programa no se ejecutará y mostrará un error de sintaxis.

- Se recomienda usar 4 espacios por nivel de indentación (no tabulaciones). Yo personalmente recomiendo tabular 1 vez.
- Todos los elementos dentro del mismo bloque deben tener la misma cantidad de espacios.
- No mezcles espacios y tabulaciones, ya que puede causar errores difíciles de detectar.

Ejemplo

Python Copiar

```python
def saludar():
    print("Hola")          # Indentado con 4 espacios
    print("¿Cómo estás?")  # Mismo nivel de indentación

edad = 10
if edad < 13:
    print("Eres un niño")  # Parte del bloque if
else:
    print("Eres un adulto")  # Parte del bloque else
print("Fin del programa")    # Fuera del bloque
```

Ver archivo Indentacion.py

Errores comunes

- Usar diferentes niveles de indentación dentro del mismo bloque.
- Mezclar espacios y tabulaciones.
- Olvidar indentar después de estructuras como *if, for, while, def, class*, etc.

Variables

A diferencia de otros lenguajes donde tienes que reservar espacio en una caja de cierto tamaño (como en C++ o Java), en Python el objeto existe por sí solo y tú solo le pones un nombre.

Python

```python
# Así de simple se crea una variable
nombre = "Frodo"  # El objeto es el texto "Frodo", la etiqueta es 'nombre'
edad = 50         # El objeto es el número 50, la etiqueta es 'edad'
```

Python es dinámicamente tipado. Esto significa que una misma variable puede cambiar de "tipo" de dato en cualquier momento. No necesitas decirle a Python que una variable es un número; él lo deduce solo.

Python

```python
puntos = 10       # Aquí es un entero (int)
puntos = "Diez"   # Ahora es una cadena (str). ¡A Python no le importa!
```

Las reglas principales al nombrar variables son:

- **Uso de snake_case**: Todo en minúsculas y separado por guiones bajos (ej: usuario_id, precio_final).
- **Empieza con letras**: No pueden empezar con números (2variable es error, variable2 está bien).
- **Sensible a mayúsculas**: Edad y edad son dos variables totalmente distintas.
- **Sin palabras prohibidas(reservadas)**: No puedes llamar a tu variable *if, else, while o print*, porque Python ya las usa para sus propias funciones.

Python permite hacer cosas muy elegantes en una sola línea:

Python

```python
# Asignar el mismo valor a muchas variables
x = y = z = 0

# Asignar valores distintos a variables distintas (Unpacking)
nombre, nivel, clase = "Aragorn", 20, "Guerrero"

# Intercambiar valores (¡Sin usar una variable temporal!)
a, b = 1, 2
a, b = b, a  # Ahora a vale 2 y b vale 1
```

Ver archivo Variables.py

Tipos de datos

- Básicos (Fundamentales)

Numeros

Python maneja diferentes tipos de números:

- **Enteros (**int**)**: Números completos, sin decimales, tanto positivos como negativos.

```python
edad = 30
saldo_negativo = -500
```

- **Flotantes (`float`)**: Números con parte decimal. Se usa el punto (.) como separador.

```python
precio = 99.95
pi = 3.14159
```

Complejos (complex): Números con una parte real y una imaginaria, denotada con una j.

```python
numero_complejo = 3 + 4j
```

Booleanos

Representan uno de dos valores: verdadero o falso. Son el resultado de comparaciones lógicas y son fundamentales para controlar el flujo de un programa (por ejemplo, en condicionales *if*).

Los valores posibles son *True* y *False* (con la primera letra en mayúscula).

Python

```python
es_mayor_de_edad = True
tiene_descuento = False

# Ejemplo de uso en una condición
if es_mayor_de_edad:
    print("Puede pasar.")
```

¡Tip importante!

En Python existen valores *truthy* y *falsy*, aunque no se les llama exactamente así de forma oficial. El concepto es el mismo: ciertos valores se evalúan como True o False cuando se usan en contextos booleanos, como en condicionales (*if*, *while*, etc.).

Valores *falsy* en Python

Estos se evalúan como False en una expresión booleana:

- None
- *False*
- 0 (de cualquier tipo numérico: 0, 0.0, 0j)
- '' (cadena vacía)
- [] (lista vacía)
- *{}* (diccionario vacío)
- *()* (tupla vacía)
- *set()* (conjunto vacío)
- *range(0)* (rango vacío)

Ejemplo:

```python
if "":
    print("Esto es truthy")
else:
    print("Esto es falsy")  # Se imprime esto
```

Valores *truthy* en Python

Todo lo que no sea falsy se considera truthy. Por ejemplo:

- Cadenas no vacías: "Hola"
- Números distintos de cero: 42, -3.14
- Listas, tuplas, diccionarios, sets con al menos un elemento
- Objetos personalizados (a menos que definan su propio comportamiento booleano)

Ejemplo:

```python
if [1, 2, 3]:
    print("Esto es truthy")  # Se imprime esto
```

Ante la duda podemos verificar como se evalua cada expresión de manera logica con la función *bool()*

Ejemplo:

```python
print(bool(""))       # False
print(bool("Hola"))   # True
print(bool([]))       # False
print(bool([0]))      # True
```

Ver archivo TruthyFalsey.py

Cadenas de Texto

Se utilizan para almacenar texto. Se pueden definir usando comillas simples ('), dobles (") o triples (''' o """) para textos de varias líneas.

Las cadenas son inmutables, lo que significa que no se pueden modificar una vez creadas (aunque sí se puede crear una nueva a partir de una existente).

Python

```python
nombre = "Ana"
saludo = 'Bienvenido a Python'
mensaje_largo = """Este es un mensaje
que ocupa varias líneas."""
```

¡Tip importante!

Un template string (o cadena plantilla) es una forma de construir cadenas de texto que permite insertar variables o expresiones directamente dentro del texto, sin tener que concatenar manualmente. Es muy útil para generar mensajes dinámicos, reportes, HTML, logs, etc. Este es una funcionalidad muy útil de python que nos acompañara habitualmente.

En Python existen unas 2 o 3 formas de usarlas(*.format(val…), operador modulo(%s, %d, %f)*), pero te daré la más legible y actualizada, se llama f-strings

Python Copiar

```python
nombre = "Ana"
edad = 30
mensaje = f"Hola, {nombre}. Tienes {edad} años."
print(mensaje)
```

Ver archivo TemplateStrings.py

Tipo nulo (NoneType)

Este tipo tiene un único valor: *None*. Se utiliza para representar la ausencia de un valor. Es similar a *null* en otros lenguajes.

Python

```python
resultado = None
# Se usa a menudo para inicializar una variable que tendrá un valor más adelante
if resultado is None:
    print("La variable aún no tiene un valor asignado.")
```

- Avanzados (Estructuras de Datos / Colecciones)

Estos tipos se utilizan para agrupar y organizar otros datos. Son extremadamente útiles y potentes.

Listas (*list*)

Son colecciones ordenadas y mutables (modificables) de elementos. Pueden contener datos de diferentes tipos. Se definen con corchetes [].

- Ordenadas: Los elementos mantienen el orden en que fueron añadidos.
- Mutables: Puedes agregar, eliminar o cambiar elementos después de crear la lista.

Python

```python
frutas = ["manzana", "banana", "cereza"]
numeros = [1, 2, 3, 4, 5]
mixta = [10, "hola", True, 20.5]

# Modificar un elemento
frutas[1] = "mango"  # Cambia "banana" por "mango"
print(frutas)        # Salida: ['manzana', 'mango', 'cereza']

# Añadir un elemento al final
frutas.append("naranja")
print(frutas)        # Salida: ['manzana', 'mango', 'cereza', 'naranja']
```

Ver archivo Listas.py

Tuplas (*tuple*)

Son colecciones ordenadas e inmutables. Una vez que se crea una tupla, no se puede modificar. Son más eficientes que las listas. Se definen con paréntesis ().

- Ordenadas: Mantienen el orden de los elementos.
- Inmutables: No puedes cambiar, agregar o eliminar elementos.

Python

```python
coordenadas = (10.0, 20.5)
colores_rgb = ("rojo", "verde", "azul")

# Intentar modificar una tupla dará un error
# coordenadas[0] = 5.0  # Esto genera un TypeError

# Se usan a menudo para datos que no deben cambiar, como coordenadas o constantes.
```

Ver archivo Tuplas.py

Diccionarios (*dict*)

Son colecciones desordenadas y mutables que almacenan datos en pares clave: valor. Son extremadamente rápidos para buscar datos por su clave. Se definen con llaves {}.

- Clave-Valor: Cada elemento tiene una clave única que se usa para acceder a su valor.
- Mutables: Puedes añadir, modificar o eliminar pares clave-valor.

Python

```python
persona = {
    "nombre": "Carlos",
    "edad": 28,
    "ciudad": "Caracas"
}

# Acceder a un valor a través de su clave
print(persona["nombre"])  # Salida: Carlos

# Modificar un valor
persona["edad"] = 29

# Añadir un nuevo par
persona["profesion"] = "Ingeniero"

print(persona)
# Salida: {'nombre': 'Carlos', 'edad': 29, 'ciudad': 'Caracas', 'profesion': 'Inge
```

Ver archivo Diccionarios.py

Conjuntos (*set*)

Son colecciones desordenadas y mutables que no permiten elementos duplicados. Son muy útiles para operaciones matemáticas de conjuntos (unión, intersección, diferencia). Se definen con llaves {} o con la función set().

- Sin duplicados: Si añades un elemento que ya existe, simplemente se ignora.
- Desordenados: No puedes estar seguro del orden en que aparecerán los elementos.

Python

```python
numeros_unicos = {1, 2, 3, 4, 4, 5, 2}
print(numeros_unicos)  # Salida: {1, 2, 3, 4, 5}

# Añadir un elemento
numeros_unicos.add(6)

# Operaciones de conjunto
otro_set = {4, 5, 6, 7, 8}
interseccion = numeros_unicos.intersection(otro_set)
print(interseccion)  # Salida: {4, 5, 6}
```

Ver archivo Sets.py

Scope (alcance o ámbito de la variable)

El "scope" o ámbito de una variable se refiere a la región del código donde esa variable es accesible. Cuando intentas acceder a una variable, Python busca en diferentes niveles de ámbito para encontrarla. Entender cómo funciona el scope es fundamental para escribir código limpio, evitar errores y manejar tus variables de manera eficiente. En definitiva entender esto nos evita volvernos un ocho al escribir nuestros programas.

Python define principalmente cuatro tipos de scopes, que se exploran en el siguiente orden (conocido como la regla LEGB):

- **Local**: El ámbito más interno, que contiene las variables definidas dentro de una función. Una variable creada dentro de una función pertenece al scope local de esa función. Solo es accesible desde dentro de esa función.

Python

```python
def mi_funcion():
    # Esta es una variable local
    animal = "Gato"
    print(f"Dentro de la función: {animal}")

mi_funcion()

# Si intentamos acceder a 'animal' fuera de la función, obtendremos un error.
try:
    print(f"Fuera de la función: {animal}")
except NameError as e:
    print(e)
```

- **Enclosing** (envolvente): El ámbito de las funciones anidadas (una dentro de la otra). Contiene las variables de la función externa, es decir la función interna puede acceder a las variables de la función externa.

Python

```python
def funcion_externa():
    # Variable en el scope 'enclosing'
    mensaje = "Hola desde fuera"

    def funcion_interna():
        # La función interna puede leer la variable de la externa
        print(mensaje)

    funcion_interna()

funcion_externa()
```

Como veremos en *global*, modificar una variable del Scope "enclosing" desde la función interna, debes usar la palabra clave *nonlocal*.

```python
    # Variable en el scope enclosing
    numero = 0

    def contador_interno():
        nonlocal numero  # Le decimos a Python que no cree una variable local
        numero += 1
        return numero

    return contador_interno

mi_contador = contador_externo()
print(mi_contador())
print(mi_contador())
print(mi_contador())
```

Como mini ejercicio, intenta adivinar el resultado de la ejecución de este codigo.

- **Global**: El ámbito a nivel de módulo. Son las variables definidas en la parte principal de tu script. Una variable declarada fuera de cualquier función tiene un scope global. Puede ser leída desde cualquier parte del código, incluyendo dentro de las funciones.

```python
# Esta es una variable global
planeta = "Tierra"

def mostrar_planeta():
    # Podemos leer la variable global desde aquí
    print(f"Desde la función, el planeta es: {planeta}")

mostrar_planeta()
print(f"Fuera de la función, el planeta es: {planeta}")
```

¿Qué pasa si quieres modificar una variable global desde una función?, si en Python se puede. Si simplemente intentas asignarle un nuevo valor, crearás una nueva variable local con el mismo nombre. Para modificarla, necesitas usar la palabra clave *global*.

Python

```python
puntuacion = 100

def actualizar_puntuacion():
    # Sin 'global', esto crearía una nueva variable local 'puntuacion'
    # global puntuacion  # Descomenta esta línea para ver la diferencia
    puntuacion = 150
    print(f"Puntuación dentro de la función: {puntuacion}")

print(f"Puntuación inicial: {puntuacion}")
actualizar_puntuacion()
print(f"Puntuación final: {puntuacion}")
```

Antes de descomentar la linea 5, intenta evaluar la salida anterior y posterior a la acción.

Built-in: Este es el nivel más alto. Contiene todas las funciones y nombres que vienen incorporados en Python y que están siempre disponibles, como len(), print(), range(), str(), etc. Python busca en este ámbito en último lugar.

Python

```python
# len es una función del scope built-in
mi_lista = [1, 2, 3, 4]
longitud = len(mi_lista)
print(f"La longitud es: {longitud}")
```

Es posible sobrescribir los nombres del scope built-in (aunque no es una buena práctica), porque Python busca primero en los scopes más internos.

Python

```python
# ¡No hagas esto en tu código real!
def mi_funcion_mala():
    # Aquí 'print' es una variable local, no la función built-in
    print = "He roto la función de imprimir"
    # La siguiente línea daría un error porque 'print' ya no es una función
    # print("Hola")

mi_funcion_mala()
```

Ver archivo Scope.py

Nombres de variables en Python

Python recomienda seguir convenciones claras para que el código sea legible y coherente; la guía de estilo oficial PEP 8 es la referencia más habitual en la comunidad. (Te invito a revisar PEP 8)

Reglas básicas de sintaxis

- Inicio válido: un nombre debe comenzar con una letra o un guion bajo _.
- Caracteres permitidos: letras, dígitos y guiones bajos; no se permiten espacios ni símbolos especiales.
- Sensibilidad a mayúsculas: Python distingue entre *mi_var* y *Mi_Var*.
- Reservadas: no usar palabras reservadas del lenguaje como *if*, *for*, *class* como nombres de variables.

Estilos comunes de nombrado

snake_case

- Descripción: palabras en minúsculas separadas por guiones bajos, por ejemplo, *mi_variable* o *calcular_total*.
- Uso: variables, funciones y métodos.

camelCase

- Descripción: primera palabra en minúscula y siguientes con la inicial en mayúscula, por ejemplo, *miVariable*.
- Uso: aceptable si se mantiene coherencia con código existente, pero no es la convención preferida para variables en Python.

PascalCase

- Descripción: cada palabra comienza con mayúscula, por ejemplo, *MiClase*.
- Uso: convención para nombres de clases en Python.

UPPER_SNAKE

- Descripción: mayúsculas con guiones bajos, por ejemplo, *MAX_INT* o *PI*.
- Uso: para constantes (valores que por convención no cambian)

Nombres especiales y prefijos

- **_privado**: un guion bajo inicial indica uso interno o “privado” dentro de un módulo o clase.
- **name**: nombres con doble guion bajo a ambos lados son usados por el lenguaje (métodos especiales, dunder) y deben reservarse para eso.
- **_ (variable única)**: a veces se usa _ para valores descartados en iteraciones o asignaciones.

Buenas prácticas rápidas

Sé descriptivo: prefiere *contador_usuarios* frente a *c* cuando sea relevante.

Coherencia: sigue el mismo estilo dentro de un proyecto.

Evita abreviaturas crípticas y usar *class_* si necesitas evitar una palabra reservada

```python
# Ejemplos
mi_variable = 10            # snake_case recomendado
MiClase = type('X', (), {})  # PascalCase para clases
MAX_INT = 100               # constante en UPPER_SNAKE
_variable_privada = 5       # uso interno
```

Estructuras de control

Las estructuras de control son los pilares de la programación. Permiten que tu programa tome decisiones y repita acciones, haciéndolo dinámico e inteligente. En Python, las estructuras de control principales son los condicionales (para tomar decisiones) y los bucles (para repetir tareas).

Condicionales

Los condicionales ejecutan un bloque de código solo si se cumple una determinada condición. Son la forma en que tu programa puede responder a diferentes situaciones

1. *If, else, elif*

El bloque if es el más básico. Ejecuta el código que contiene solo si su condición es verdadera (True).

Sintaxis

```python
if condicion:
    # Código a ejecutar si la condición es verdadera
```

Ejemplo

Python

```python
edad = 19

if edad >= 18:
    print("Eres mayor de edad. ¡Puedes pasar!")
```

A menudo, quieres que el programa haga algo diferente si la condición no se cumple. Para eso se usa *else*.

Sintaxis

Python

```python
if condicion:
    # Código a ejecutar si la condición es verdadera
else:
    # Código a ejecutar si la condición es falsa
```

Ejemplo

Python

```python
edad = 15

if edad >= 18:
    print("Eres mayor de edad. ¡Puedes pasar!")
else:
    print("Eres menor de edad. No puedes pasar.")
```

Cuando tienes más de dos posibilidades, puedes encadenar múltiples condiciones usando *elif*. Python las evaluará en orden y ejecutará el bloque de la primera condición que sea verdadera.

Ejemplo

Python

```python
nota = 85

if nota >= 90:
    print("Calificación: A")
elif nota >= 80:
    print("Calificación: B")
elif nota >= 70:
    print("Calificación: C")
else:
    print("Calificación: Reprobado")
```

Ver archivo IfElseElif.py

2. Bucles *while* y *for*

Los bucles se utilizan para repetir un bloque de código varias veces, ya sea un número determinado de veces o mientras se cumpla una condición.

El bucle *while* repite un bloque de código mientras una condición sea verdadera. Es ideal cuando no sabes exactamente cuántas veces necesitas repetir el ciclo.

Sintaxis

Python

```python
while condicion:
    # Código a repetir
```

Ejemplo

```python
contador = 1

while contador <= 5:
    print(f"Número: {contador}")
    contador += 1  # ¡Muy importante! Incrementar el contador para evitar un bucle

print("Fin del bucle while.")
```

Ver archivo While.py

El bucle *for* se utiliza para iterar sobre una secuencia de elementos (como una lista, una tupla, un diccionario, un string o un rango). Es perfecto cuando quieres ejecutar un bloque de código para cada elemento de una colección.

Sintaxis

```python
for variable in secuencia:
    # Código a ejecutar para cada elemento
```

Ejemplo 1: Iterar sobre una lista(array)

```python
frutas = ["manzana", "banana", "cereza"]

for fruta in frutas:
    print(f"Me gusta comer {fruta}")
```

Ejemplo 2: Iterar sobre una tupla

```python
mi_tupla = ('manzana', 'banana', 'cereza')

for fruta in mi_tupla:
    print(fruta)
```

Ejemplo 3: Iterar sobre un diccionario

Recuerda que un diccionario por su constitución no es un objeto iterable.

```python
mi_diccionario = {'nombre': 'Ana', 'edad': 30, 'ciudad': 'Madrid'}

for clave in mi_diccionario:
    print(clave)
```

También puedes usar *mi_diccionario.keys()* explícitamente, entendiendo que el método *keys()* regresa las claves del diccionario:

```python
for clave in mi_diccionario.keys():
    print(clave)
```

Lo mismo aplica para los valores, con el método *values()*:

```python
for valor in mi_diccionario.values():
    print(valor)
```

De igual manera podemos iterar sobre las claves-valores directamente con

```python
for clave, valor in mi_diccionario.items():
    print(f"{clave}: {valor}")
```

Ejemplo 4: Iterar sobre una cadena de texto

Python Copiar

```python
texto = "Python"

for letra in texto:
    print(letra)
```

Ejemplo 5: Usando *range()* para repetir N veces

Python

```python
# range(5) genera números del 0 al 4
for i in range(5):
    print(f"Repetición número {i}")
```

A veces necesitas un mayor control sobre cómo se ejecuta un bucle. Para eso tenemos las sentencias *break y continue.* La declaración break detiene y sale del bucle inmediatamente, sin importar si la condición del bucle sigue siendo verdadera.

Python

```python
numeros = [1, 5, 12, 18, 23, 30]

for numero in numeros:
    print(f"Comprobando {numero}...")
    if numero == 18:
        print("¡Lo encontré!")
        break  # Salimos del bucle
```

La declaración *continue* omite la iteración actual y salta directamente a la siguiente.

Python

```python
for i in range(1, 11):  # Números del 1 al 10
    if i % 2 == 0:  # Si el número es par...
        continue    # ...saltamos a la siguiente iteración.
    print(i)
```

Ver el archivo For.py

Bonus!, La sentencia *match*

La sentencia *match* es una forma moderna y poderosa de manejar múltiples condiciones. Es similar al *switch* en otros lenguajes, pero con capacidades mucho más avanzadas gracias al emparejamiento de patrones.

Permite comparar un valor contra varios patrones y ejecutar el bloque de código que coincida. Es útil para simplificar estructuras *if/elif/else* complejas y puede trabajar con:

- valores simples (números, cadenas)
- estructuras como listas, tuplas o diccionarios
- condiciones adicionales dentro de cada caso

Sintaxis

Python Copiar

```python
match variable:
    case valor1:
        # Código si variable == valor1
    case valor2:
        # Código si variable == valor2
    case _:
        # Código si no coincide ningún caso (comodín)
```

Ejemplo

Python Copiar

```python
def clasificar_dia(dia):
    match dia:
        case "lunes" | "martes" | "miércoles" | "jueves" | "viernes":
            return "Es un día laboral"
        case "sábado" | "domingo":
            return "Es fin de semana"
        case _:
            return "No es un día válido"

print(clasificar_dia("domingo"))  # Imprime: Es fin de semana
```

Ver archivo Match.py

1.5 Características, casos de uso, aplicaciones e introducción a lenguajes estructurados de alto nivel.

Un Vistazo a Otros Lenguajes de Alto Nivel

¿Por qué hay tantos lenguajes de programación?

Así como un carpintero tiene diferentes herramientas para diferentes trabajos (un martillo para clavar, una sierra para cortar), los programadores utilizan diferentes lenguajes de programación según el problema que necesiten resolver. Cada lenguaje está diseñado con ciertas fortalezas, características y "filosofías" que lo hacen ideal para tareas específicas.

Aunque en este curso nos centramos en Python, es fundamental conocer otros lenguajes populares para entender mejor el panorama tecnológico actual. A continuación, exploraremos algunos de los más influyentes.

1. JavaScript: El Lenguaje de la Web

Si Python es el rey de la ciencia de datos y el backend, JavaScript es el rey indiscutible del desarrollo web del lado del cliente (frontend).

Breve Historia

Creado en 1995 por Brendan Eich en Netscape, fue diseñado en solo 10 días con el objetivo de añadir interactividad a las páginas web, que hasta entonces eran estáticas. Hoy en día, es un pilar fundamental de la web moderna.

Características Clave:

- **Lenguaje Interpretado y Dinámico**: Similar a Python, el código se ejecuta directamente en el navegador sin necesidad de compilación previa. Los tipos de las variables pueden cambiar durante la ejecución.
- **Orientado a Eventos**: Su principal fortaleza es la capacidad de responder a las acciones del usuario, como clics, movimientos del ratón o pulsaciones de teclas (onClick, onMouseOver, etc.).
- **Multiplataforma por Naturaleza**: Funciona en cualquier dispositivo que tenga un navegador web.
- **Ecosistema Enorme (Node.js):** Con la llegada de Node.js, JavaScript rompió las barreras del navegador y ahora también se puede usar en el backend, permitiendo crear servidores, APIs y aplicaciones de escritorio.

Aplicaciones y Casos de Uso:

- **Desarrollo Frontend**: Crear la interactividad y dinamismo de casi todas las páginas web que visitas (menús desplegables, animaciones, validación de formularios). Frameworks como React, Angular y Vue.js dominan este campo.

- **Desarrollo Backend**: Construir servidores web rápidos y escalables con Node.js.
- **Aplicaciones Móviles**: Usando frameworks como React Native, se puede escribir código en JavaScript que funciona tanto en iOS como en Android.
- **Aplicaciones de Escritorio**: Con herramientas como Electron (usado por Visual Studio Code, Slack y Discord).

2. Java: "Escribe una vez, ejecuta en cualquier lugar"

Java es un gigante del mundo empresarial y de las aplicaciones a gran escala. Su lema ("Write Once, Run Anywhere" - WORA) define su principal ventaja.

Breve Historia

Desarrollado por James Gosling en Sun Microsystems y lanzado en 1995. Fue creado con la idea de ser un lenguaje robusto, seguro y, sobre todo, portable.

Características Clave

- **Compilado e Interpretado**: El código Java se compila primero a un formato intermedio llamado bytecode. Este bytecode es luego interpretado por la Máquina Virtual de Java (JVM), que se puede instalar en cualquier sistema operativo (Windows, macOS, Linux). Esto es lo que le da su portabilidad.
- **Fuertemente Tipado y Orientado a Objetos (OOP):** A diferencia de Python o JavaScript, en Java debes declarar explícitamente el tipo de cada variable (*int numero = 10;).* Su diseño se basa estrictamente en los principios de la Programación Orientada a Objetos.
- **Robusto y Seguro**: Incluye manejo de memoria automático (Garbage Collection) y mecanismos de seguridad que lo hicieron popular para aplicaciones empresariales.
- **Gran Ecosistema y Madurez**: Posee una cantidad inmensa de librerías, herramientas y frameworks probados durante décadas para todo tipo de tareas.

Aplicaciones y Casos de Uso

- **Desarrollo de Aplicaciones para Android**: Es el lenguaje nativo e histórico para crear aplicaciones en el sistema operativo móvil más grande del mundo.
- **Sistemas Empresariales a Gran Escala**: Bancos, aseguradoras y grandes corporaciones confían en la robustez de Java para sus sistemas de backend (por ejemplo, usando el framework Spring).
- **Aplicaciones de Big Data**: Herramientas como Hadoop y Kafka, fundamentales en el ecosistema de Big Data, están escritas en Java.

- **Aplicaciones de Escritorio**: Aunque menos común hoy en día, sigue siendo usado para software multiplataforma de escritorio.

3. C++: Potencia y Control al Máximo Nivel

Cuando el rendimiento es la máxima prioridad y se necesita un control absoluto sobre el hardware del sistema, C++ es el lenguaje a elegir.

Breve Historia

Creado por Bjarne Stroustrup en 1983 como una extensión del legendario lenguaje C, al que le añadió capacidades de Programación Orientada a Objetos.

Características Clave

- **Lenguaje Compilado**: El código C++ se traduce directamente a código máquina nativo del procesador. Esto lo hace extremadamente rápido, pero el programa compilado solo funcionará en la arquitectura para la que fue compilado (ej. un .exe de Windows no corre en Linux).
- **Control de Bajo Nivel**: Permite la manipulación directa de la memoria a través de punteros, lo que da un poder inmenso pero también una gran responsabilidad (una mala gestión puede causar graves errores).
- **Multi-paradigma**: Soporta programación estructurada, orientada a objetos y funcional.
- **Rendimiento Extremo**: Es uno de los lenguajes más rápidos que existen, solo superado por C y ensamblador.

Aplicaciones y Casos de Uso

- **Videojuegos**: Los motores gráficos más potentes del mercado, como Unreal Engine, están construidos en C++. Se necesita su velocidad para renderizar gráficos complejos en tiempo real.
- **Sistemas Operativos y Controladores (Drivers):** Partes de Windows, macOS y Linux están escritas en C++.
- **Software de Alto Rendimiento**: Navegadores web (Chrome, Firefox), bases de datos, aplicaciones de edición de video (Adobe Premiere) y software de ingeniería (CAD) dependen de la velocidad de C++.
- **Sistemas Embebidos y IoT:** Dispositivos con recursos limitados, como los controladores de un coche o electrodomésticos inteligentes, a menudo usan C++.

4. C#: La Respuesta de Microsoft a Java

C# (pronunciado "C-sharp") es un lenguaje moderno y versátil creado por Microsoft, que combina la robustez de C++ con la simplicidad de lenguajes como Java.

Breve Historia

Fue desarrollado por Microsoft a principios de los 2000 como parte de su plataforma .NET. Su objetivo era ser el lenguaje principal para desarrollar aplicaciones en el ecosistema de Windows.

Características Clave

- **Moderno y Versátil**: Al igual que Java, es fuertemente tipado y orientado a objetos, pero ha ido incorporando características modernas de otros lenguajes que lo hacen muy agradable para programar.
- **Integrado con el Ecosistema .NET**: Funciona sobre el framework .NET, que proporciona una vasta biblioteca de clases para desarrollar casi cualquier tipo de aplicación.
- **Multiplataforma**: Aunque nació ligado a Windows, gracias a .NET Core (ahora simplemente .NET), hoy en día se pueden crear y ejecutar aplicaciones de C# en Windows, macOS y Linux.

Aplicaciones y Casos de Uso

- **Desarrollo de Videojuegos**: Es el lenguaje principal del motor Unity, uno de los más populares del mundo para crear juegos para PC, consolas y dispositivos móviles.
- **Aplicaciones de Escritorio para Windows**: Sigue siendo la opción preferida para crear software de escritorio nativo para Windows (WPF, Windows Forms).
- **Desarrollo Web Backend**: Con ASP.NET, es una alternativa potente a Java y Node.js para construir servicios web y APIs robustas.
- **Aplicaciones Empresariales y en la Nube**: Es ampliamente utilizado en el entorno corporativo, especialmente en empresas que utilizan la nube de Microsoft, Azure.